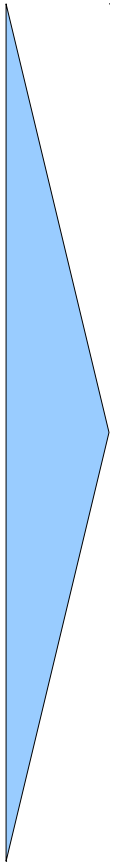
Advanced Micro-Controller Systems



- Introduction, Microcontroller basics
- 32-bit Microcontroller technology
- ARM Cortex M Architecture,(CM0 CM3 CM4 CM7)
- Cortex M Microcontroller Instruction Set
- Exception Handling, Interrupts
- Real-time Operating Systems (RTOS)
- Memory Management, protected memory, MPU
- Implementing Calculations, Data Formats, FPU
- I/O, Device drivers
- Application Design
- Case Studies, Advanced Design Patte

https://skript.eit.h-da.de/ams/

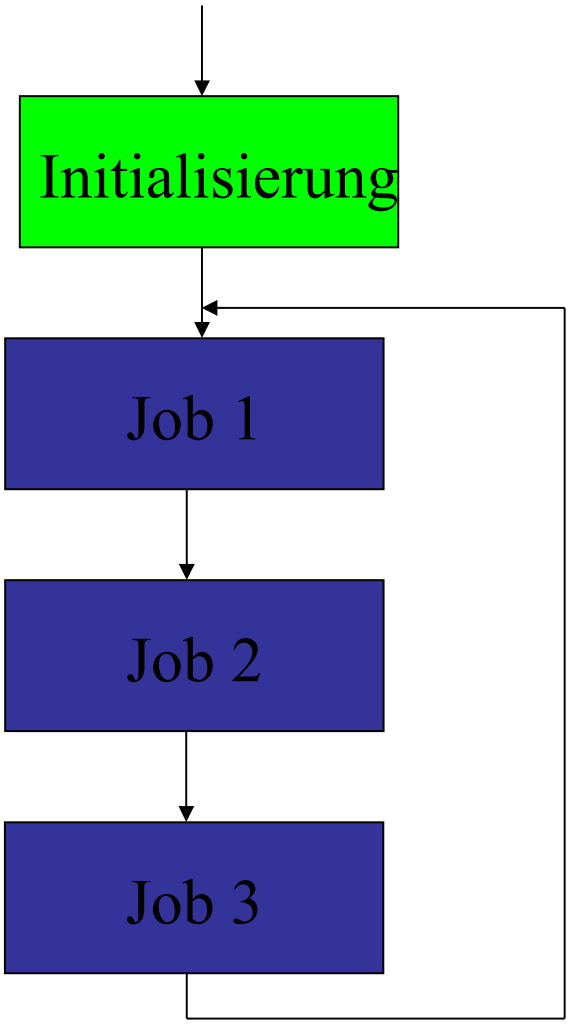
Real-time Operating Systems
RTOS

Multi-Threading
Thread-Switching

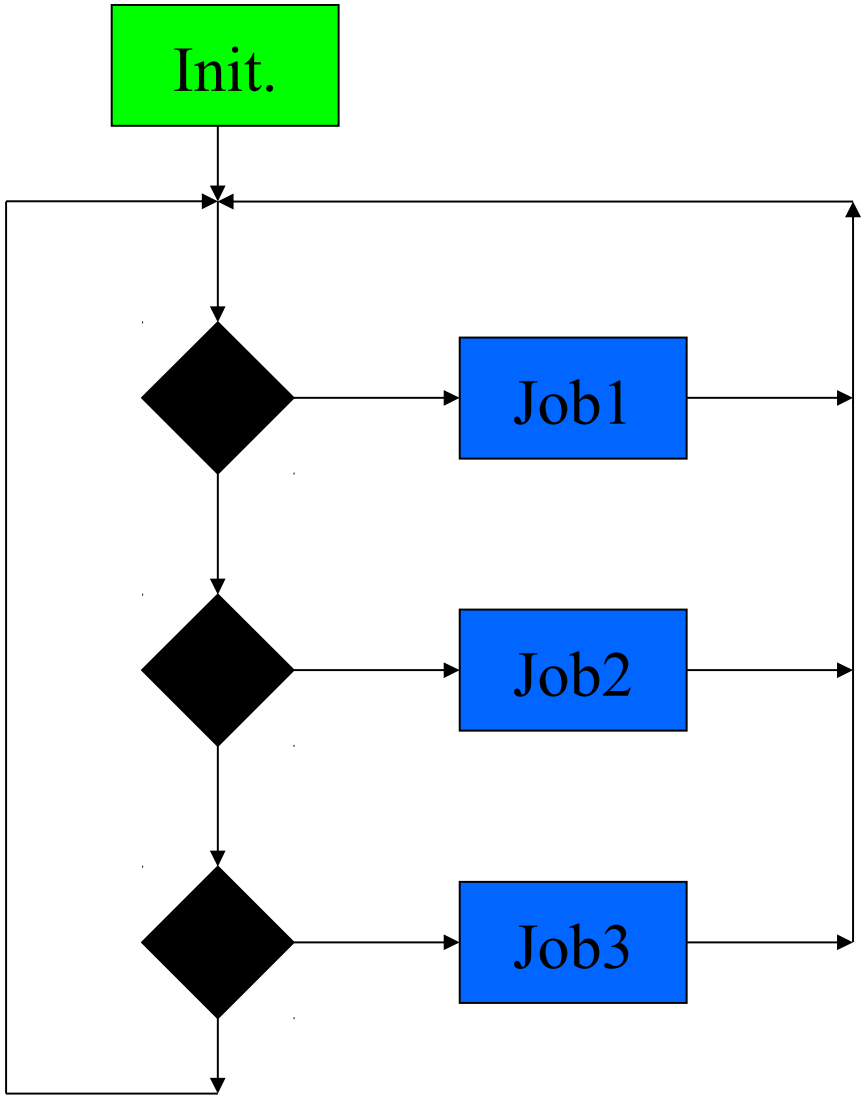
Inter - Thread Synchronisation
Inter - Thread Communication

Deadlock / Lifelock
Scheduling strategies

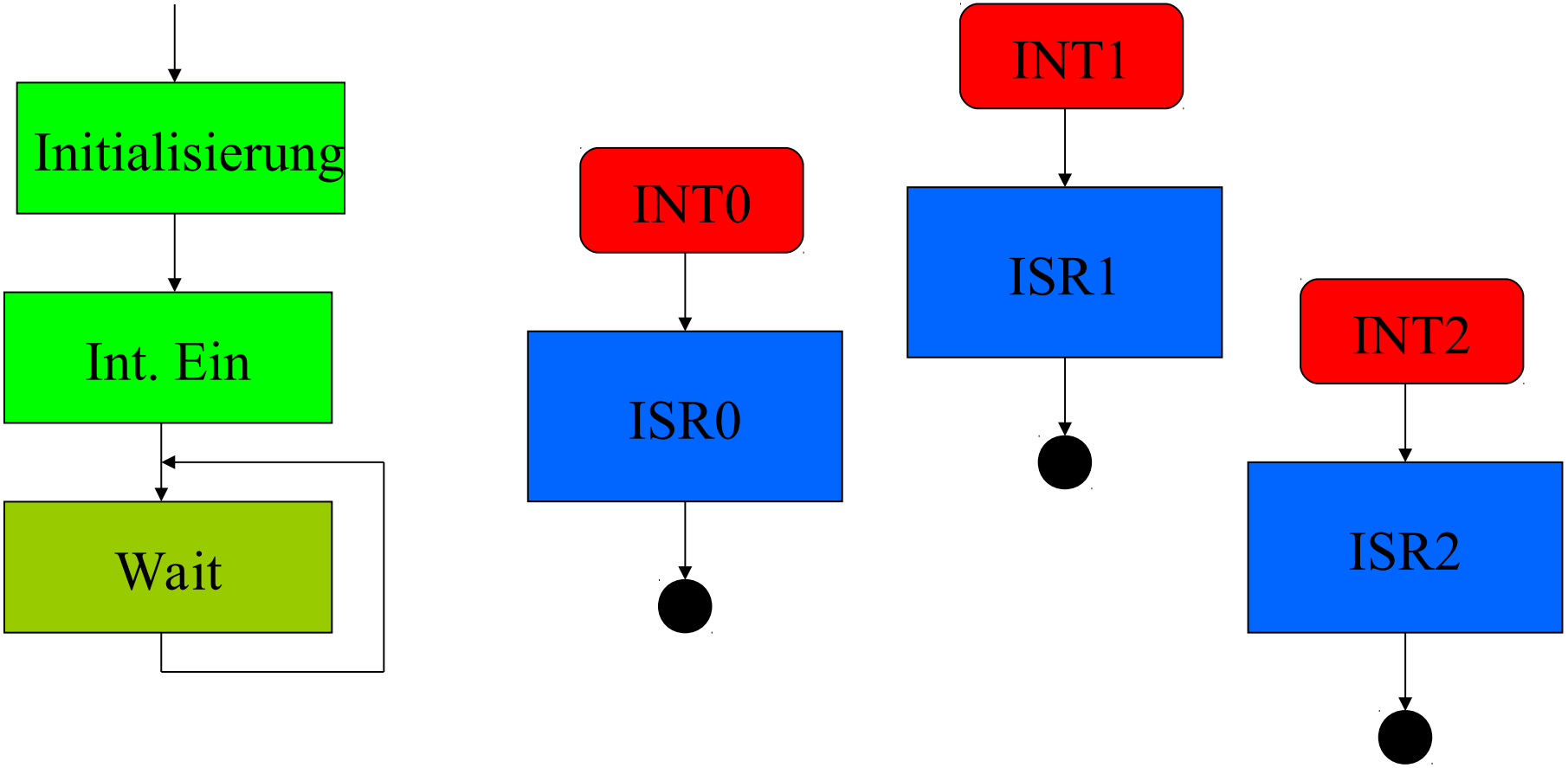
Superloop, Round Robin



Superloop, Priority-driven

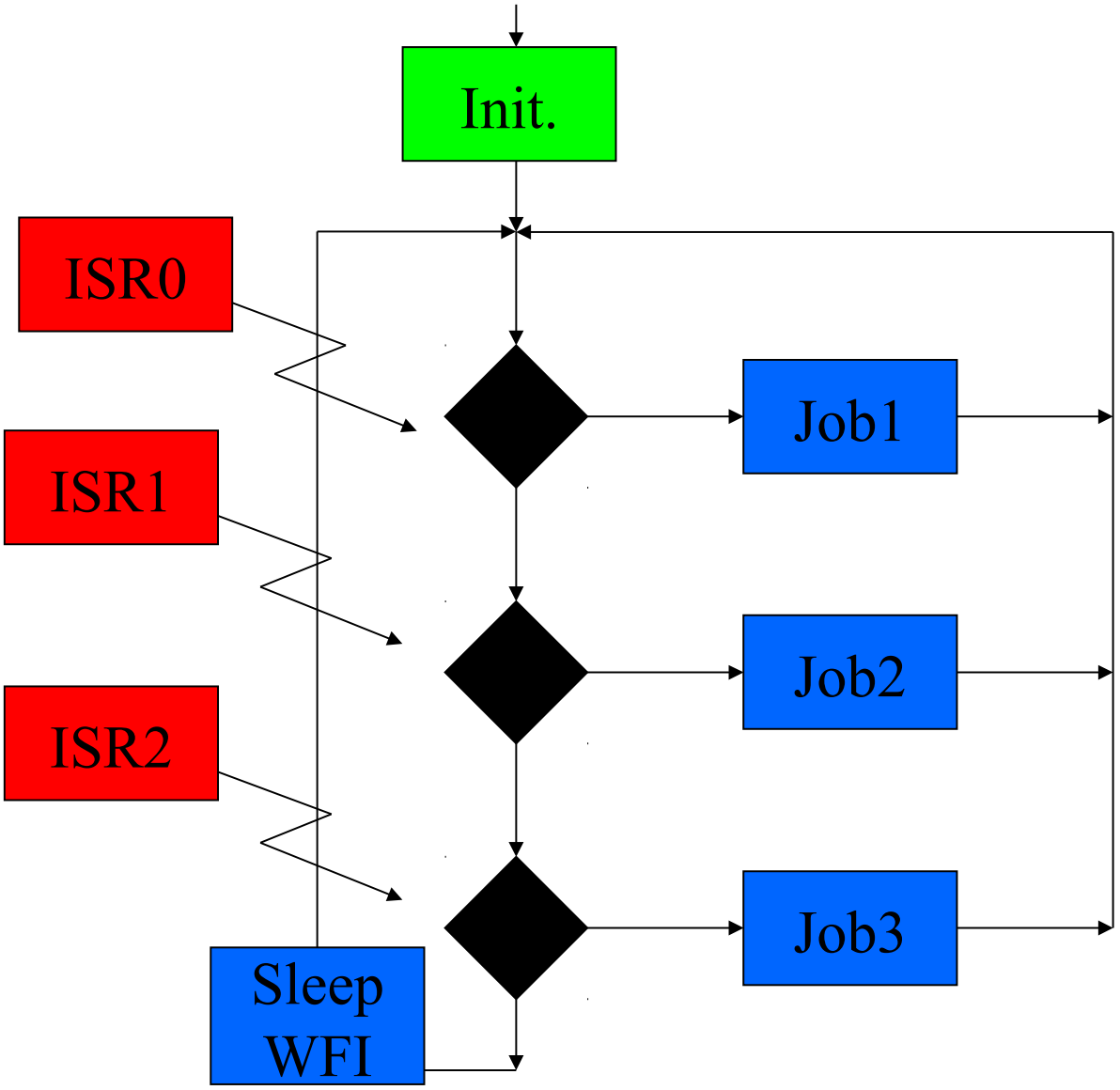


Interrupt Only

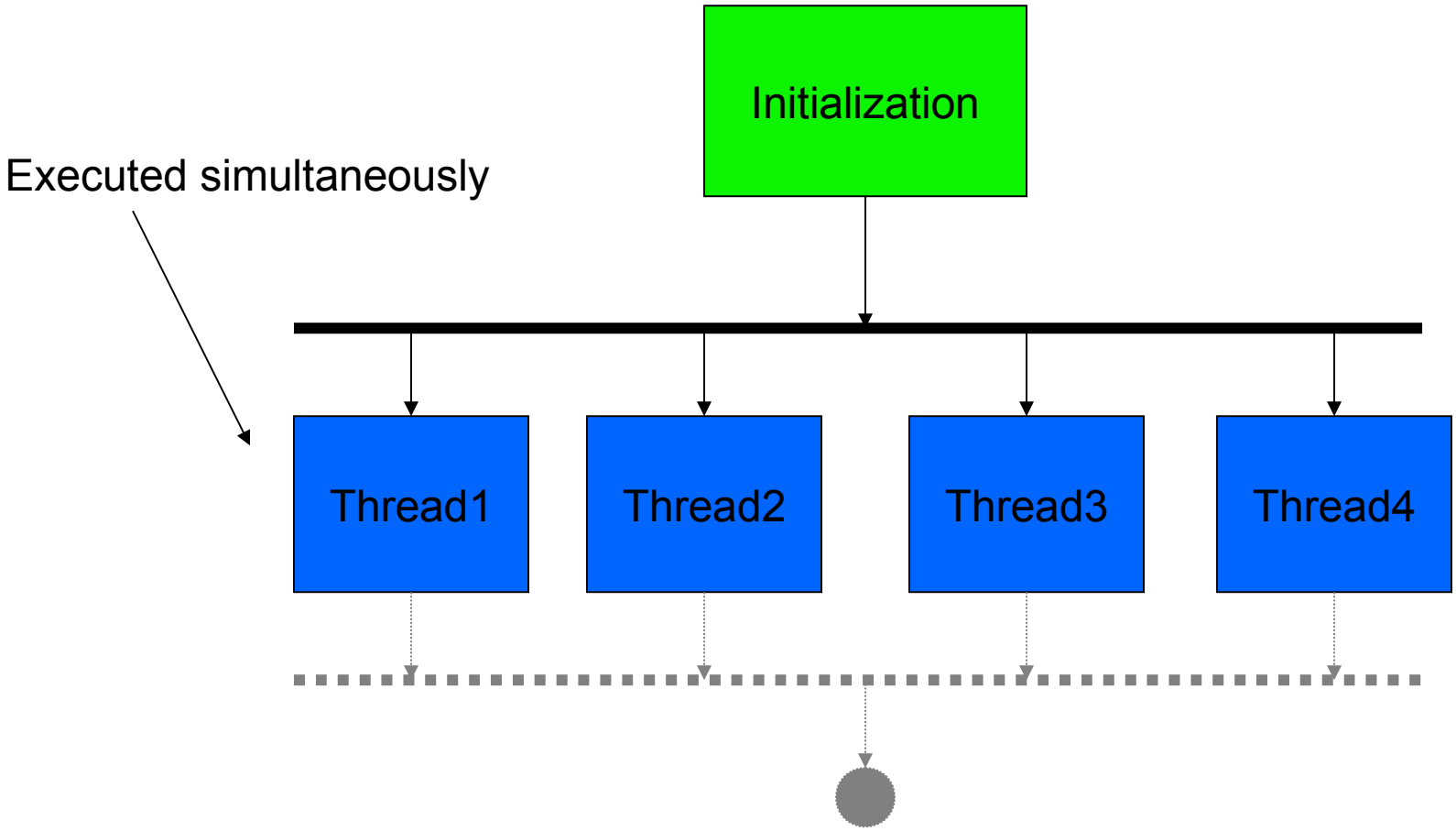


See sleep-on-exit bit within the System control Register (SCR)

Superloop + Interrupts



Multi-Threading
Multi-Tasking



Def.: Task = Thread

Process := Adressing Area on Systems with MMU

Task

- **Task (computers)**
- **From Wikipedia, the free encyclopedia**

- A **task** is "an execution path through address space". In other words, a set of program instructions that is loaded in memory. The address registers have been loaded with the initial address of the program.
  At the next clock cycle, the CPU will start execution, in accord with the program. The sense is that some part of 'a plan is being accomplished'. As long as the program remains in this part of the address space, the task can continue, in principle, indefinitely, unless the program instructions contain a halt, exit, or return.

Thread

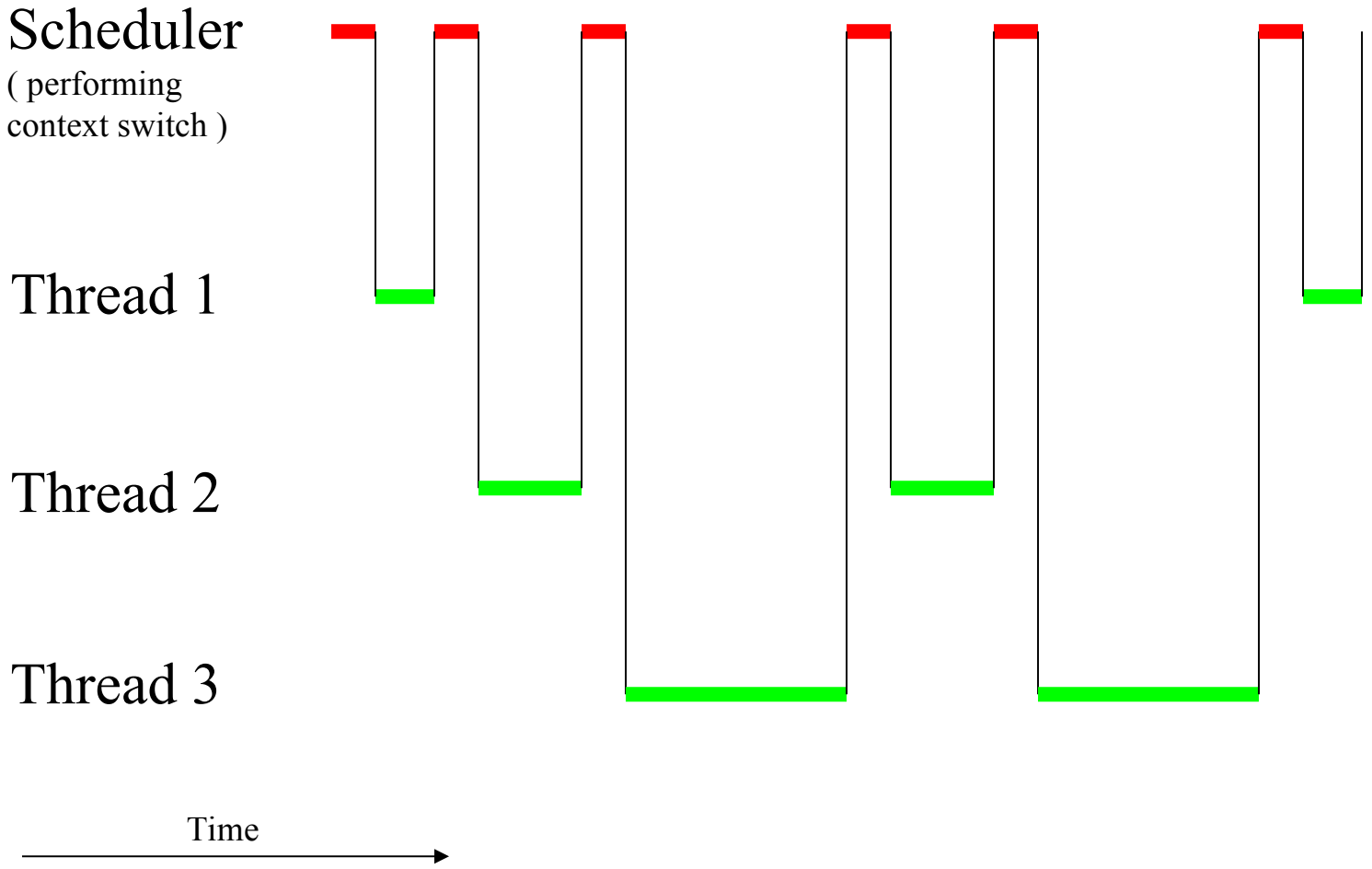
- **Thread (computer science)**
- **From Wikipedia, the free encyclopedia**

- A **thread** in computer science is short for a *thread of execution*. Threads are a way for a program to split itself into two or more simultaneously (or pseudo-simultaneously) running tasks. Threads and processes differ from one operating system to another, but in general, the way that a thread is created and shares its resources is different from the way a process does.

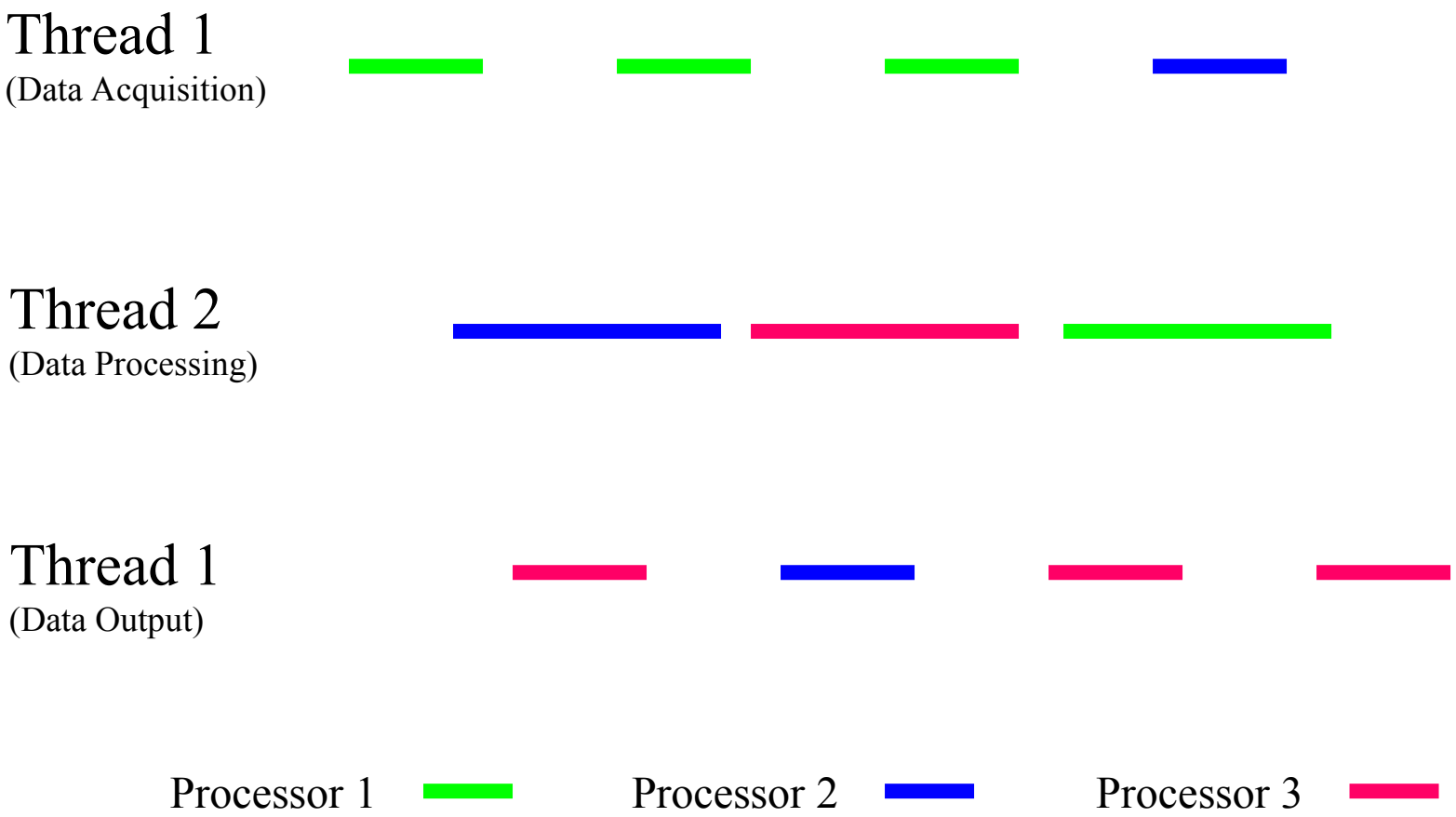
Process

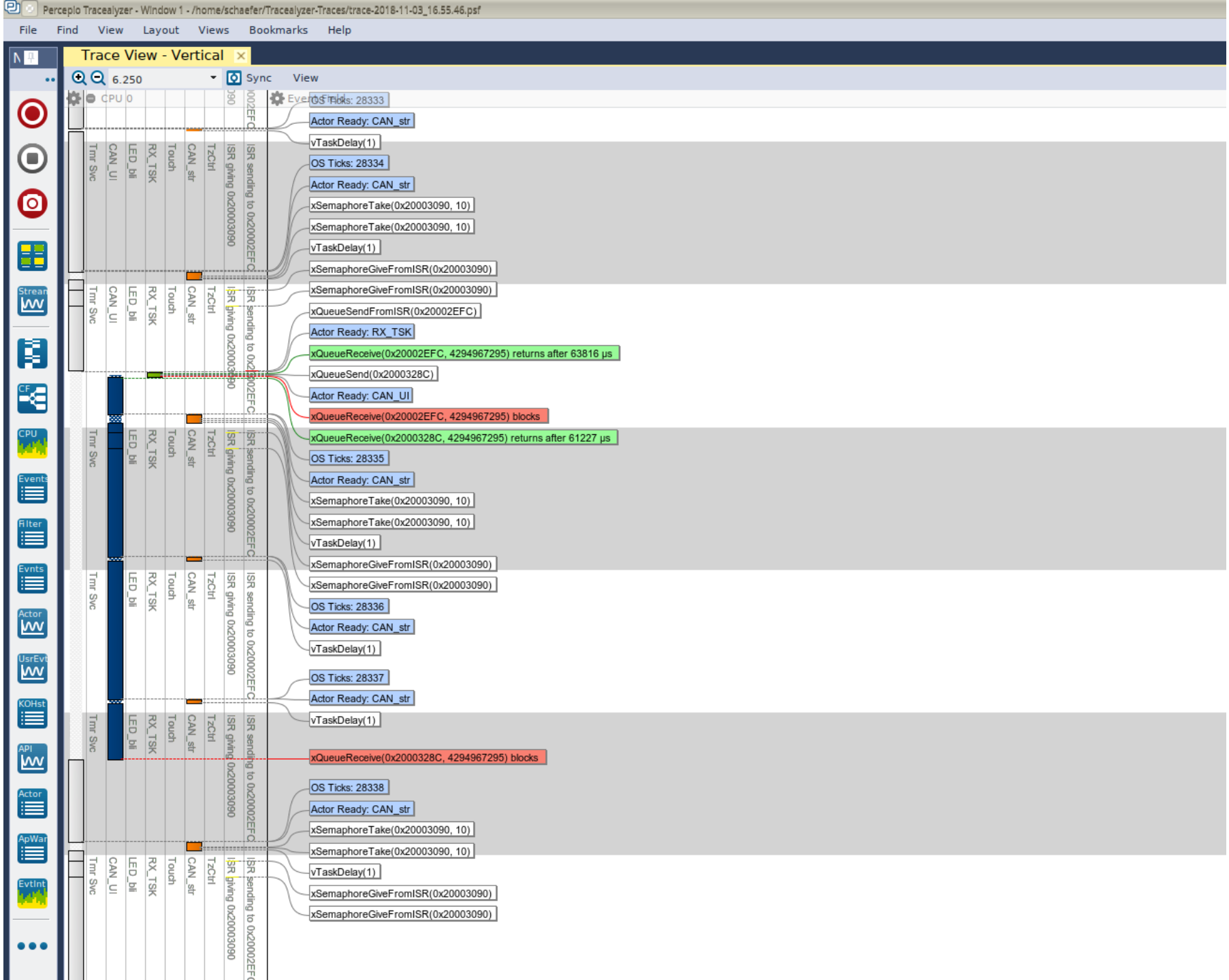
- **From Wikipedia, the free encyclopedia**
- In computing, a **process** is a running instance of a program, including all variables and other state. A modern computer system typically supports many processes, which are multitasked to give the appearance that all of them can run simultaneously.

Thread / Task -Switching



Parallel Processing
Multiple Processors




Percepio Tracealyzer - Window 1 - /home/schaefer/Tracealyzer-Traces/trace-2018-11-03_16.55.46.psf
File
Find
View
Layout
Views
Bookmarks
Help
Trace View - Vertical
6.250
Sync
View
CPU 0
Event Label
OS Ticks: 28333
Actor Ready: CAN_str
vTaskDelay(1)
OS Ticks: 28334
Actor Ready: CAN_str
xSemaphoreTake(0x20003090, 10)
xSemaphoreTake(0x20003090, 10)
vTaskDelay(1)
xSemaphoreGiveFromISR(0x20003090)
xSemaphoreGiveFromISR(0x20003090)
xQueueSendFromISR(0x20002EFC)
Actor Ready: RX_TSK
xQueueReceive(0x20002EFC, 4294967295) returns after 63816 µs
xQueueSend(0x2000328C)
Actor Ready: CAN_UI
xQueueReceive(0x20002EFC, 4294967295) blocks
xQueueReceive(0x2000328C, 4294967295) returns after 61227 µs
OS Ticks: 28335
Actor Ready: CAN_str
xSemaphoreTake(0x20003090, 10)
xSemaphoreTake(0x20003090, 10)
vTaskDelay(1)
xSemaphoreGiveFromISR(0x20003090)
xSemaphoreGiveFromISR(0x20003090)
OS Ticks: 28336
Actor Ready: CAN_str
vTaskDelay(1)
OS Ticks: 28337
Actor Ready: CAN_str
vTaskDelay(1)
xQueueReceive(0x2000328C, 4294967295) blocks
OS Ticks: 28338
Actor Ready: CAN_str
xSemaphoreTake(0x20003090, 10)
xSemaphoreTake(0x20003090, 10)
vTaskDelay(1)
xSemaphoreGiveFromISR(0x20003090)
xSemaphoreGiveFromISR(0x20003090)
Tmr Svc
CAN_UI
LED_bli
RX_TSK
Touch
CAN_str
TzCtrl
ISR giving 0x20003090
ISR sending to 0x20002EFC

Processor Context

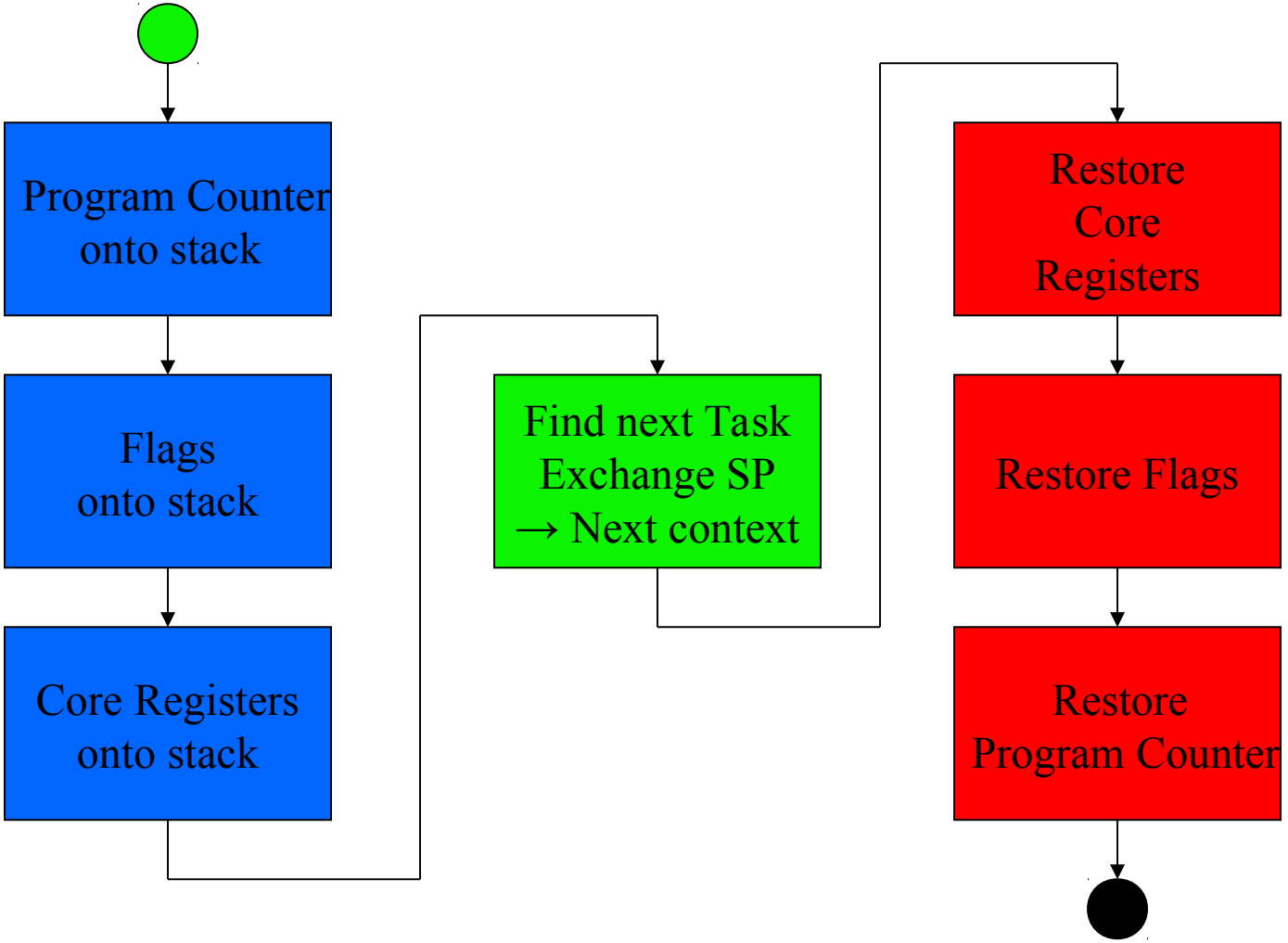
CPU execution state

Set of all register contents including:

- Program Counter
- Stackpointer
- Universalregister (i.e. R0..R12)
- Flags (i.e. PSR)
- Link Register R14

- FPU Registers S0..S31 (if in use)

Thread Switch / Context Switch



Context-Switch Cortex M3

```
void xPortPendSVHandler( void )
{
    /* This is a naked function. */

    __asm volatile
    (
    "    mrs r0, psp                         \n"
    "    isb                                 \n"
    "                                        \n"
    "    ldr r3, pxCurrentTCBConst           \n"
    "    ldr r2, [r3]                        \n"
    "                                        \n"
    "    stmdb r0!, {r4-r11}                 \n"
    "    str r0, [r2]                        \n"
    "                                        \n"
    "    stmdb sp!, {r3, r14}                \n"
    "    mov r0, %0                          \n"
    "    msr basepri, r0                     \n"
    "    bl vTaskSwitchContext               \n"
    "    mov r0, #0                          \n"
    "    msr basepri, r0                     \n"
    "    ldmia sp!, {r3, r14}                \n"
    "                                        \n"
    "    ldr r1, [r3]                        \n"
    "    ldr r0, [r1]                        \n"
    "    ldmia r0!, {r4-r11}                 \n"
    "    msr psp, r0                         \n"
    "    isb                                 \n"
    "    bx r14                              \n"
    "                                        \n"
    "    .align 2                            \n"
    "pxCurrentTCBConst: .word pxCurrentTCB   \n"
    ::"i"(configMAX_SYSCALL_INTERRUPT_PRIORITY)
    );
}
```

Push context

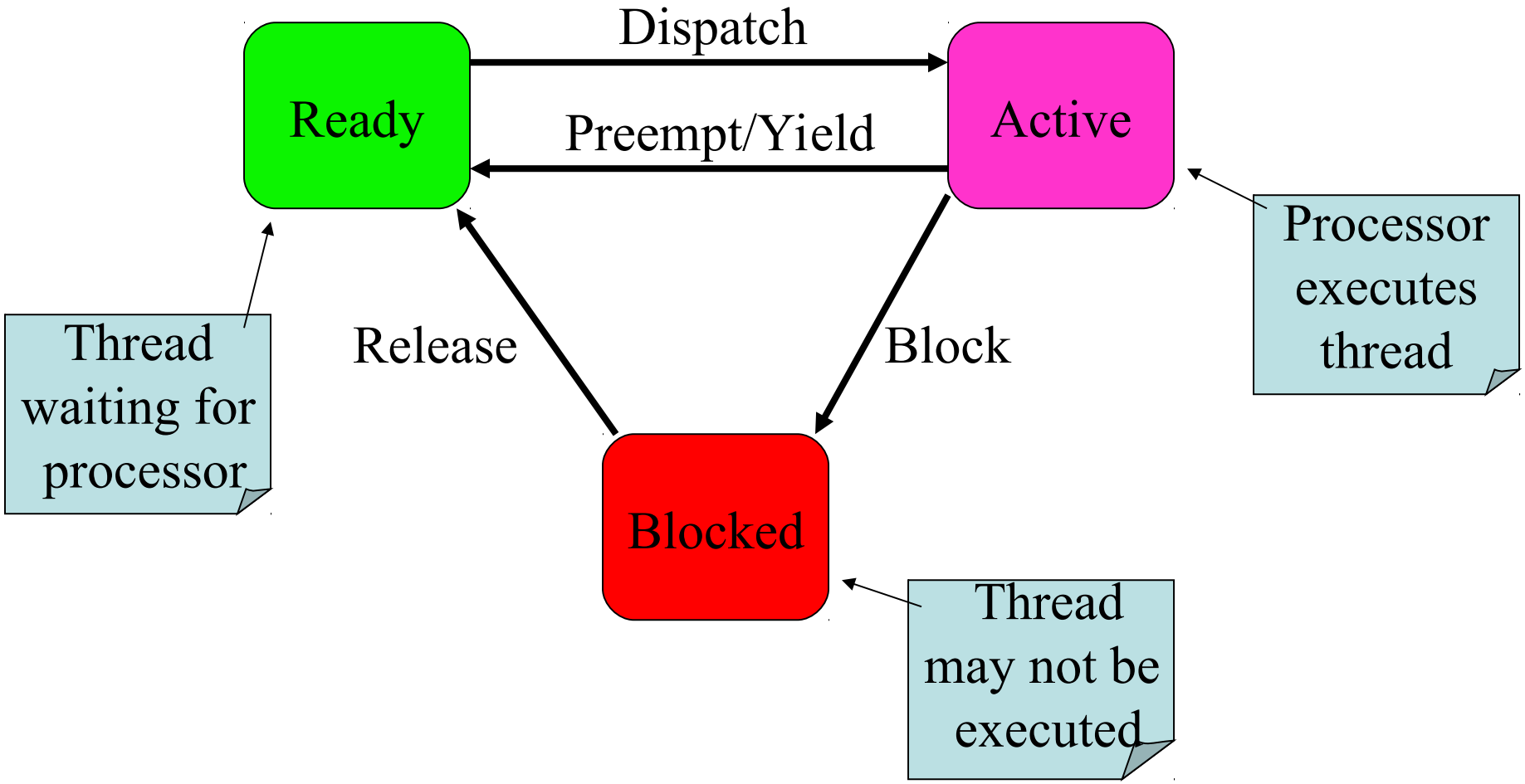
Find next task

Pop context

Load next SP

Return into task

Thread State



Thread / Task Properties

- State (Blocked / Ready)
- Priority (High … Low)
- CPU Registers, PC, SP, Flags
- Stack Content
- Addressable Memory,
  inherited from its process

Thread Switching Scheduling Policy

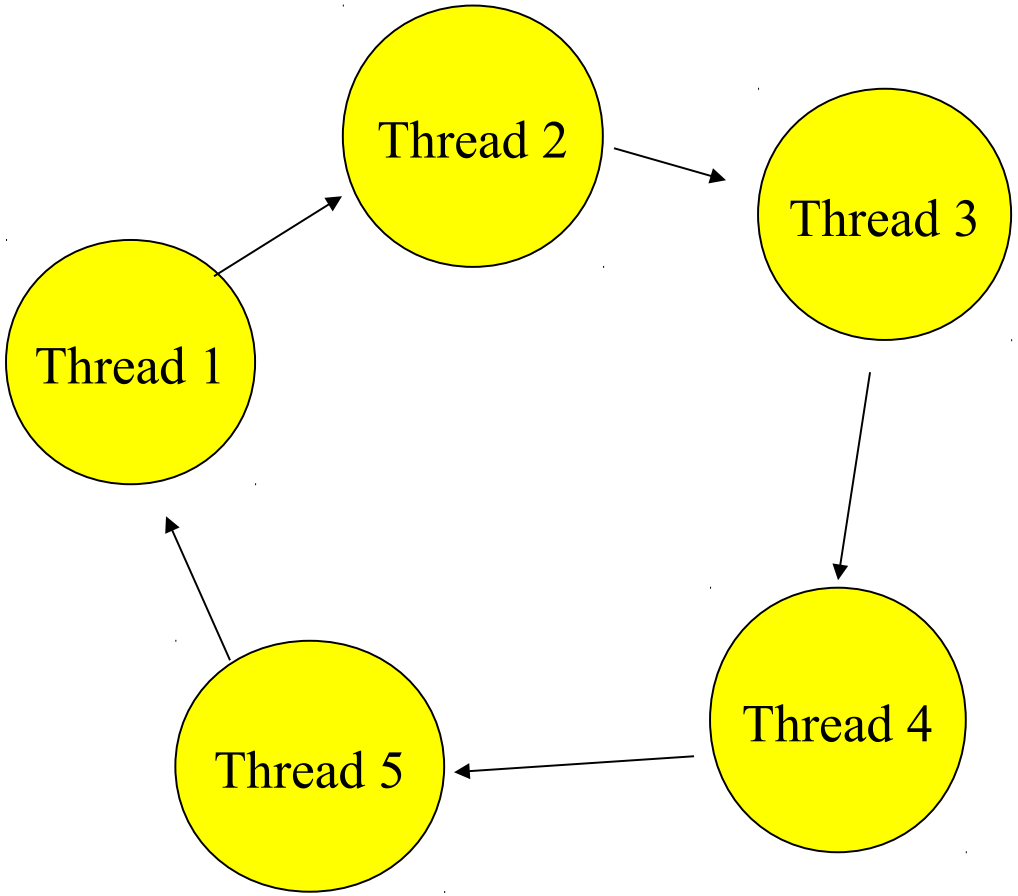
Scheduling

- Round Robin Scheduling
- Prioritzed Scheduling
- Rate Monotonic Scheduling
- Earliest Deadline First Scheduling
- Shortest Job First Scheduling
- Etc.

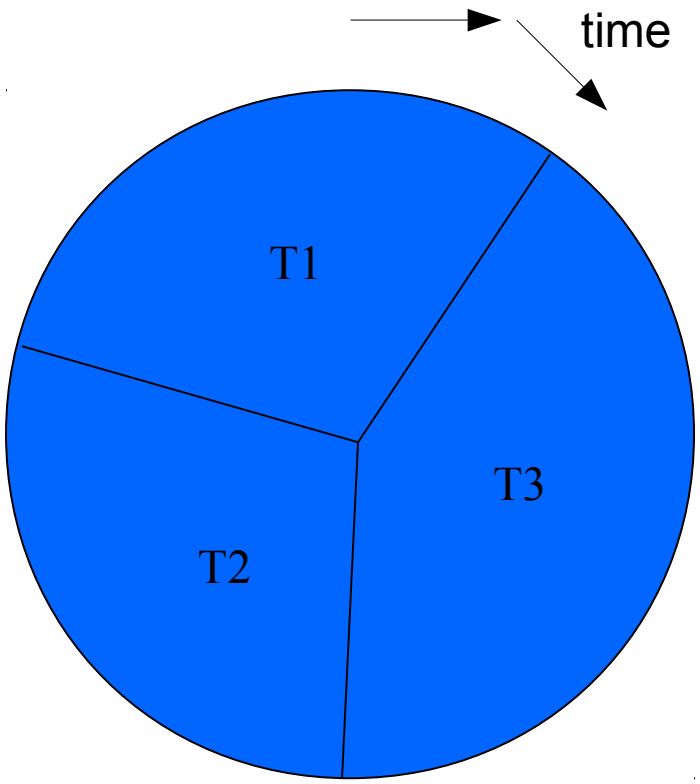
Thread Switch

- Cooperative Switching
- Preemptive Switching

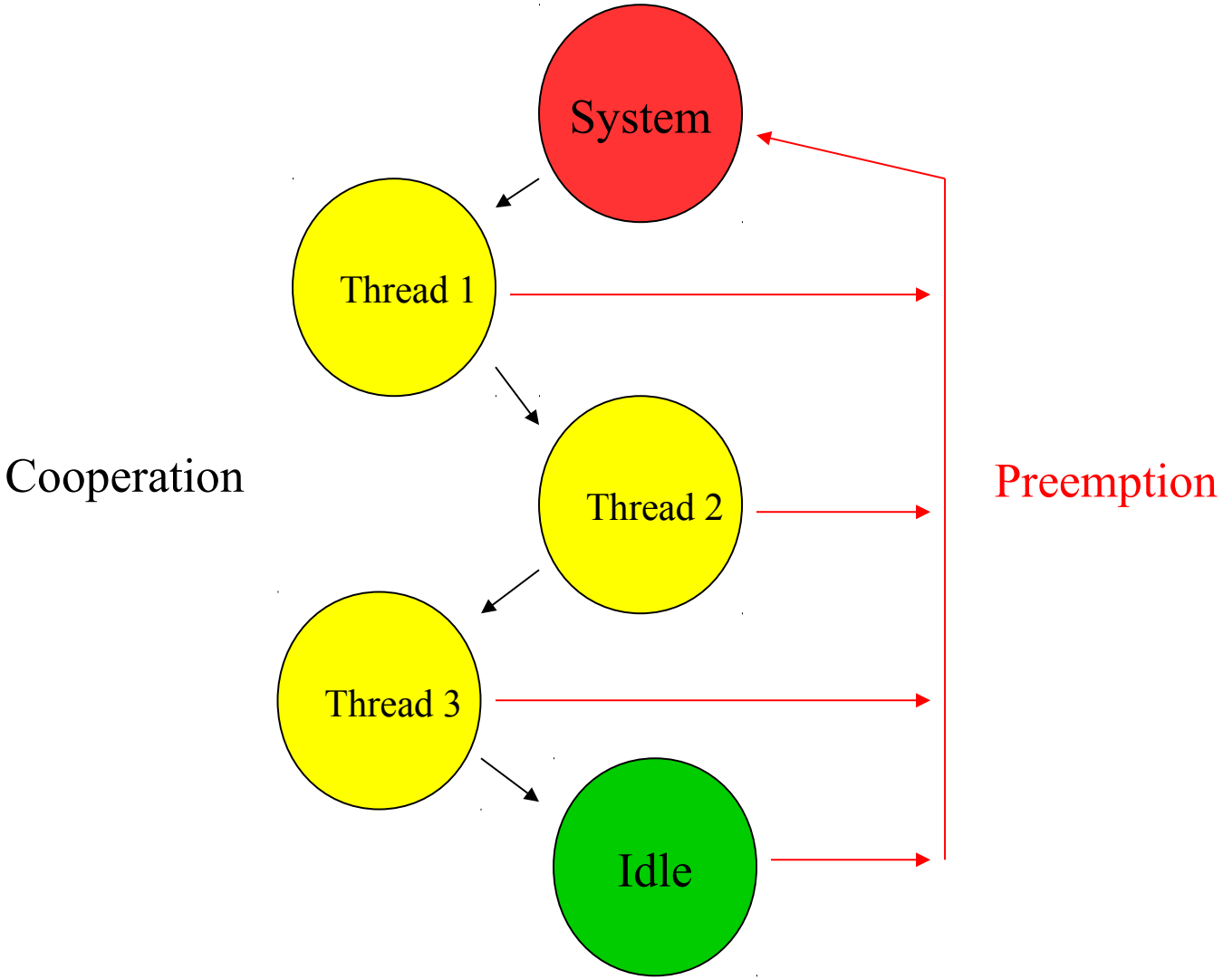
Round Robin Scheduling



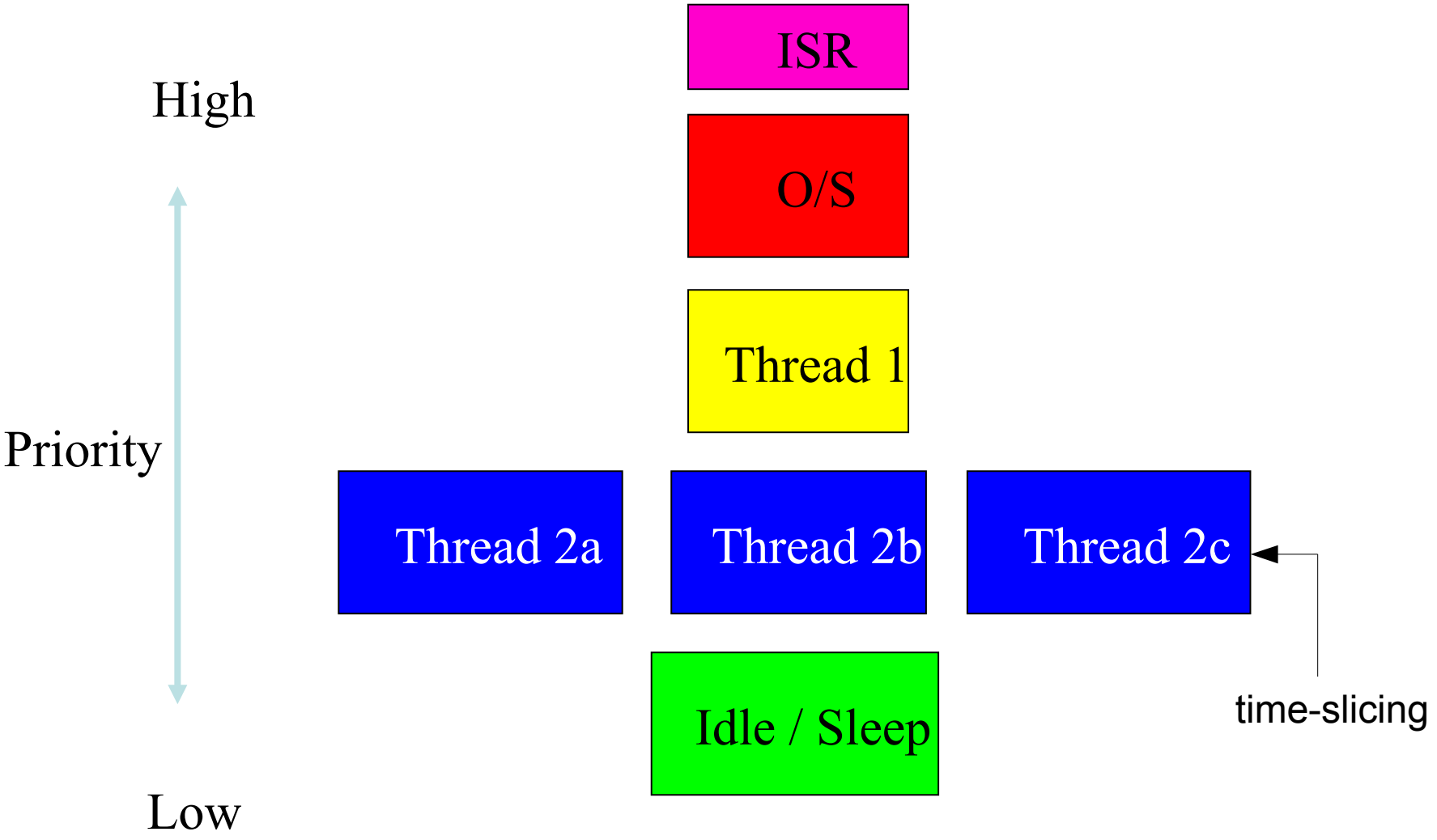
Time Sharing



Prioritized Scheduling



Priority-driven Scheduling



Cooperative Multi-Threading

Thread-switcher (RTOS) switches context

yield() must be used to hand over the MPU
from task to task:
… when the MPU is no longer needed at the moment
… to share MPU time with other tasks

Cooperative Multi-Threading

- Threads are not preempted
- Programmer has to care about reaction times
- Yield() required
- Small switching overhead
- Resource locking not required

Preemptive Multi-Threading

- RTOS enforces thread-switches with ISR‘s
- Threads do not need to call yield()
- Interrupts must be coded to allow thread switches
- Data-Race Conditions / Starvation may happen

Preemptive Multi-Threading

- Does not allow threads to monopolize the processor
- Thread priority system can be established
- Predictable response time
- Locking mechanisms necessary

Reentrant

A computer program or routine is described as **reentrant** if it can be safely called recursively or from multiple processes.

To be reentrant, a function

- must hold no static data,
- must not return a pointer to static data,
- must work only on the data provided to it by the caller, and
- must not call non-reentrant functions.

(wikipedia.org)

Reentrancy

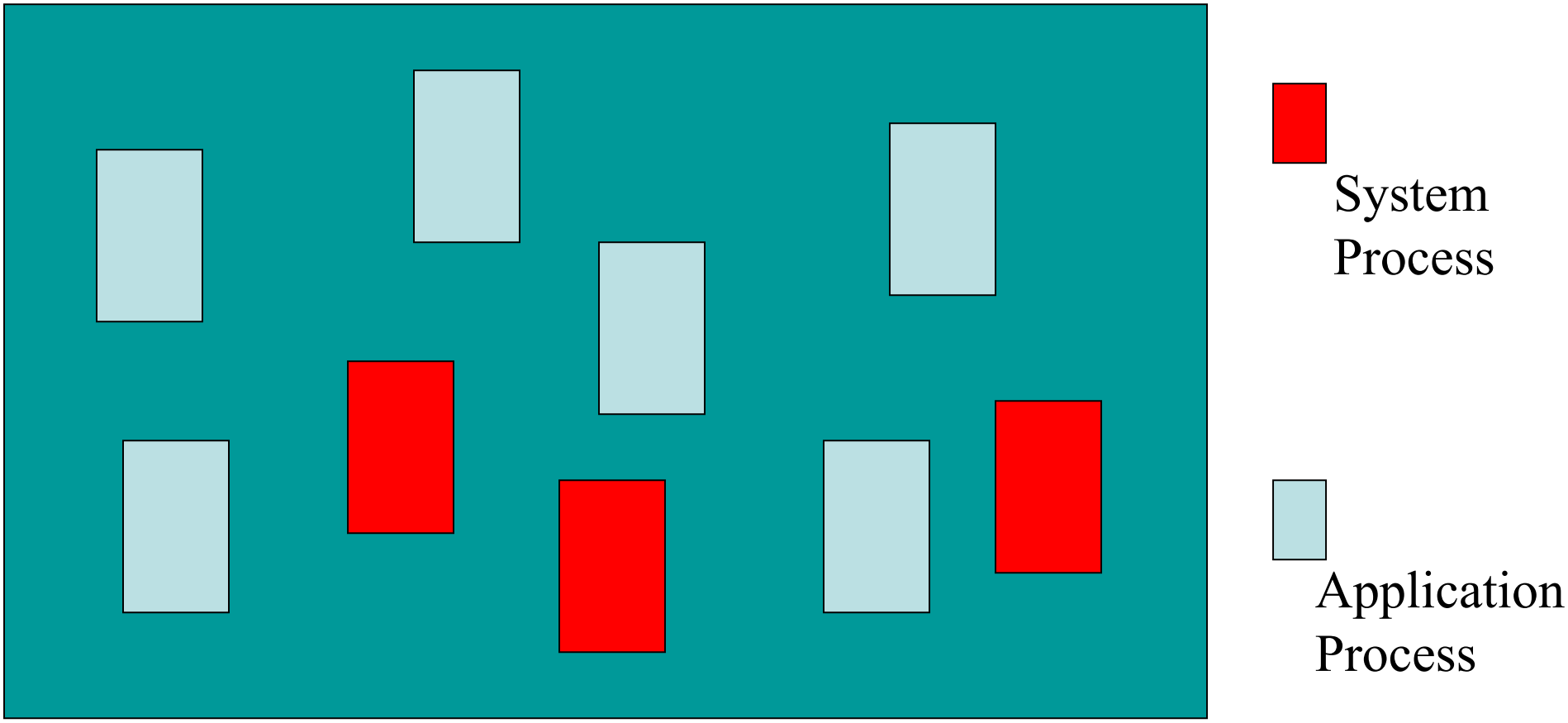
Property of a subroutine

If a function is reentrant, it can be used arbitraryly
by different tasks and ISR‘s without any interference.

Conditions to be reentrant include:

- No statical variables used
- No hardware used
- Local data only in processor registers and on the stack
  („auto“ - variables support this automatically)

Multi Process System

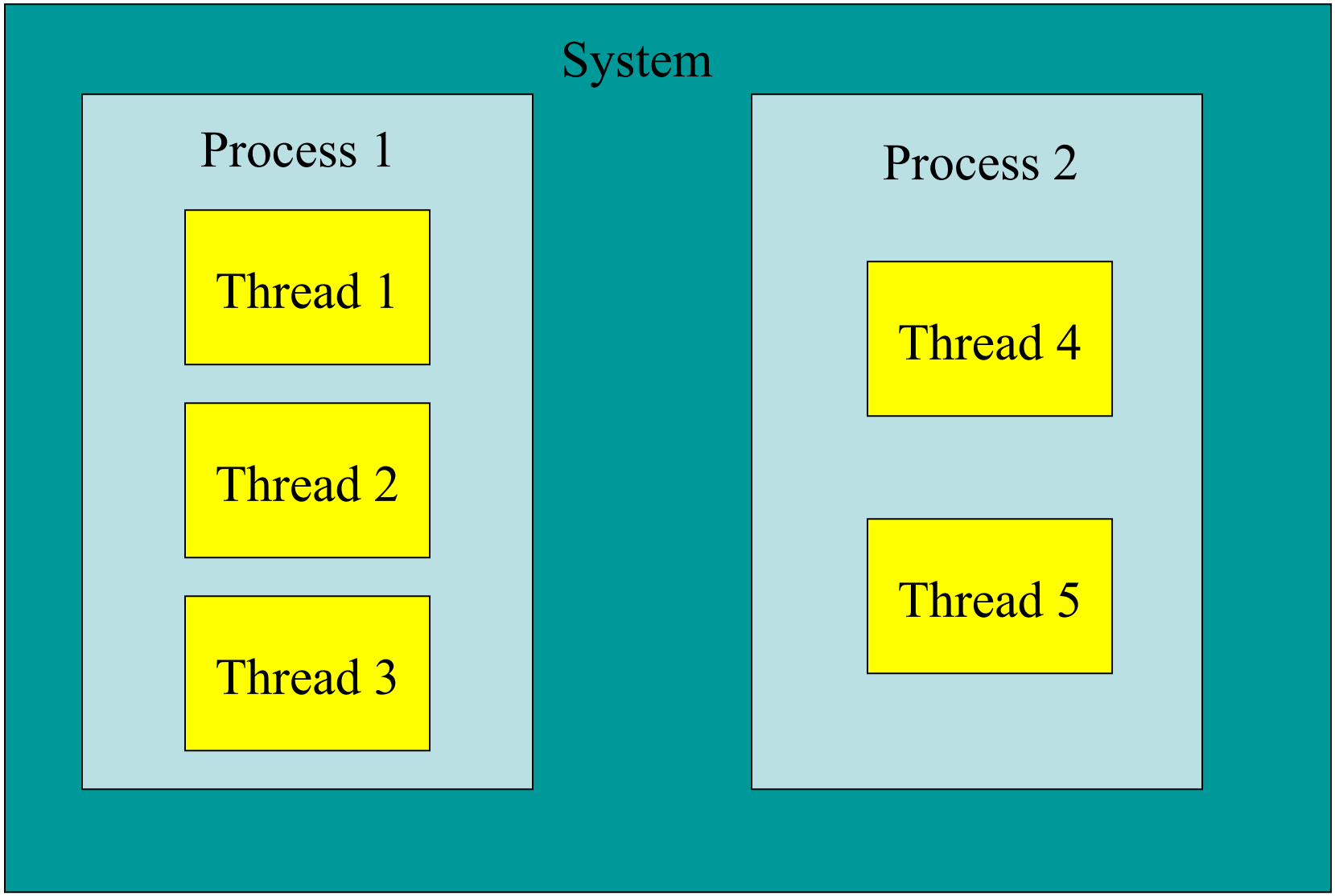


Process Properties

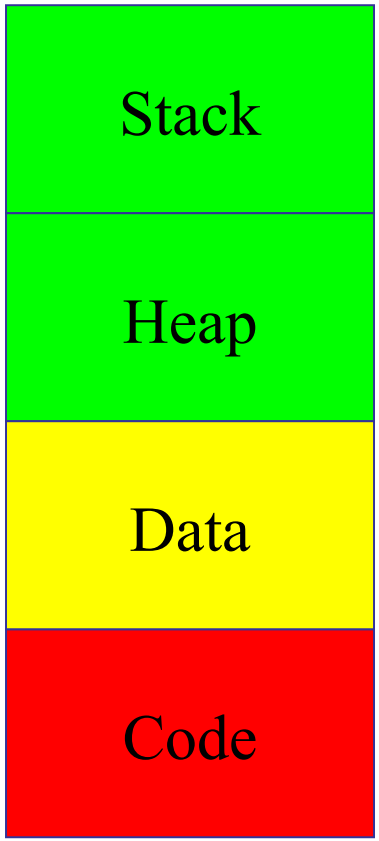
- Address Space 
- Process Identifier
- System Resources
- Environment Tables
- Security Attributes
- Thread(s)

The system is responsible for the isolation of processes

Processes and Threads



Memory Layout



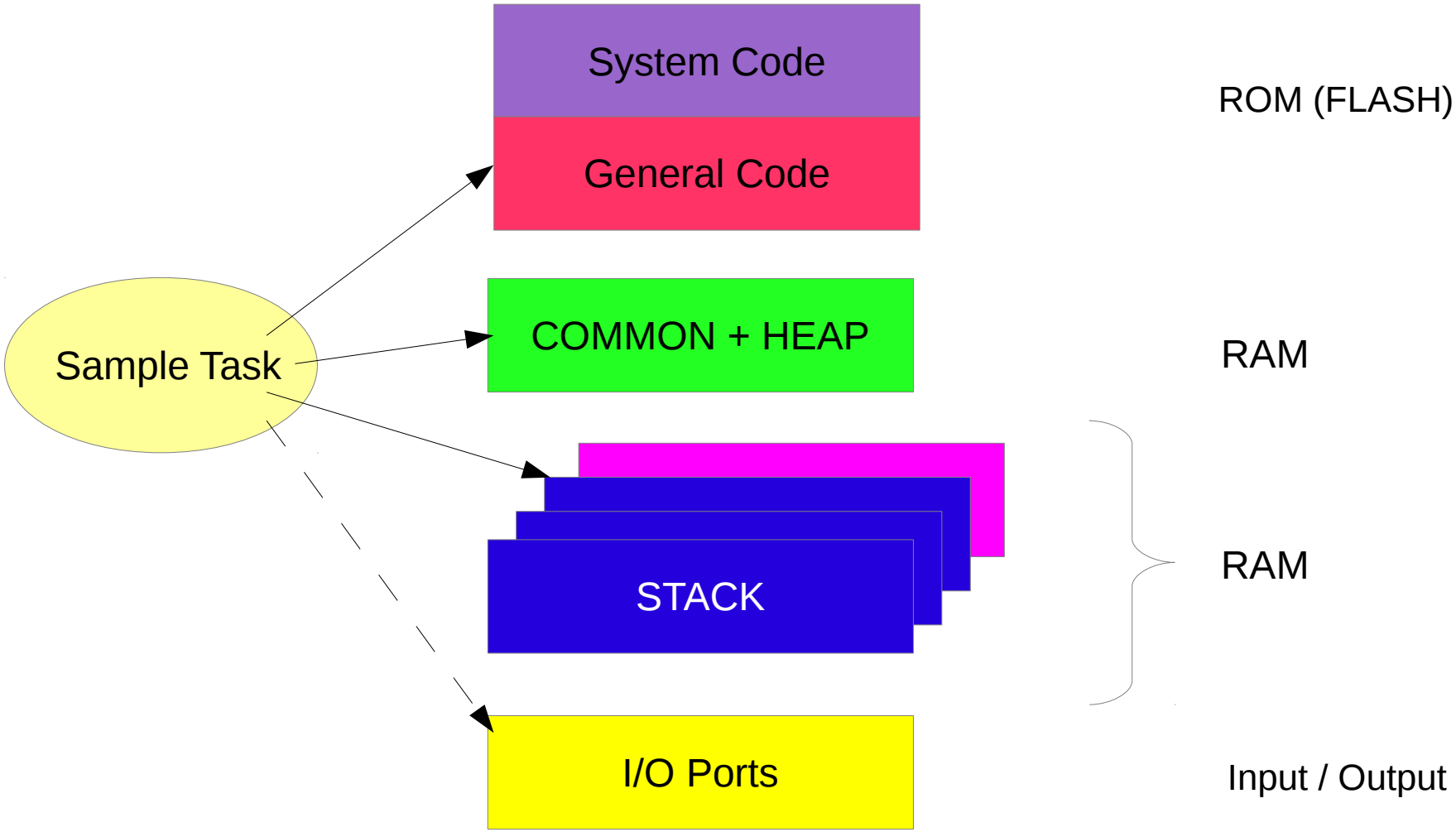
Distinct for every thread:
Subroutine Return Address, Local Data

Common:
Dynamic Data [new() / delete()]

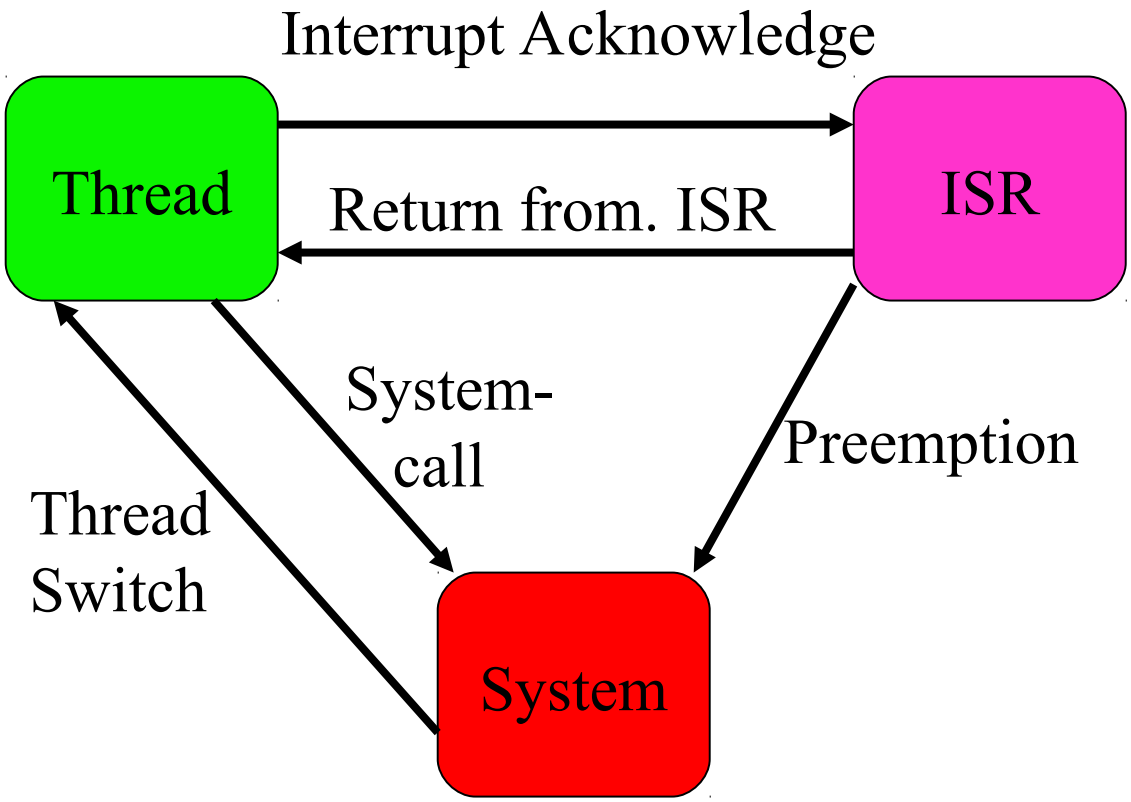
Fixed Data (static or global)

Code, const Data

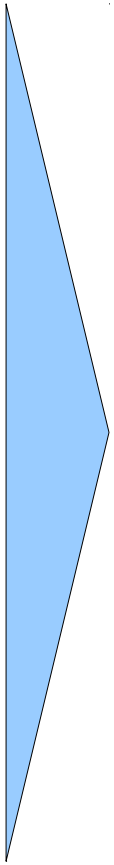
Memory Layout
Logical view



System State



Lecture Overview



- Introduction, Microcontroller basics
- 32-bit Microcontroller technology
- ARM Cortex M Architecture, (CM0 CM3 CM4 CM
- Cortex M Microcontroller Instruction Set
- Exception Handling, Interrupts
- Real-time Operating Systems (RTOS)
- Memory Management, protected memory, MPU
- Implementing Calculations, Data Formats, FPU
- I/O, Device drivers
- Application Design
- Case Studies, Advanced Design Patte

https://skript.eit.h-da.de/ams/